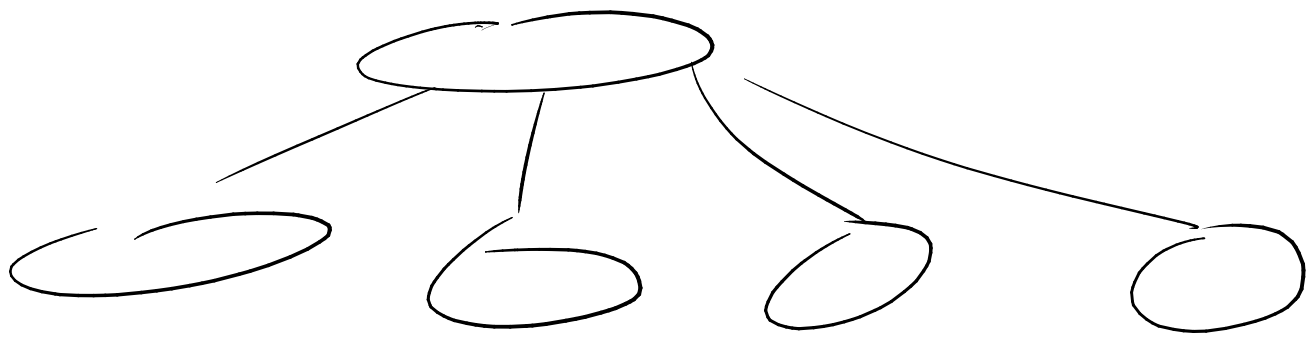
B+ trees! - most popular DB indexing technique

- pays attention to pages



each node is a page.

~4K (depending on system)

- tree is balanced
- every node has to be at least full (except for root)

Leaves and internal nodes are different

Leaves just contain search keys and "pointers" [RIDs] to actual data

Each leaf is a page. How many search keys and RIDs can it store?

In example ⟶

each id is an int (4 bytes, if a short?)

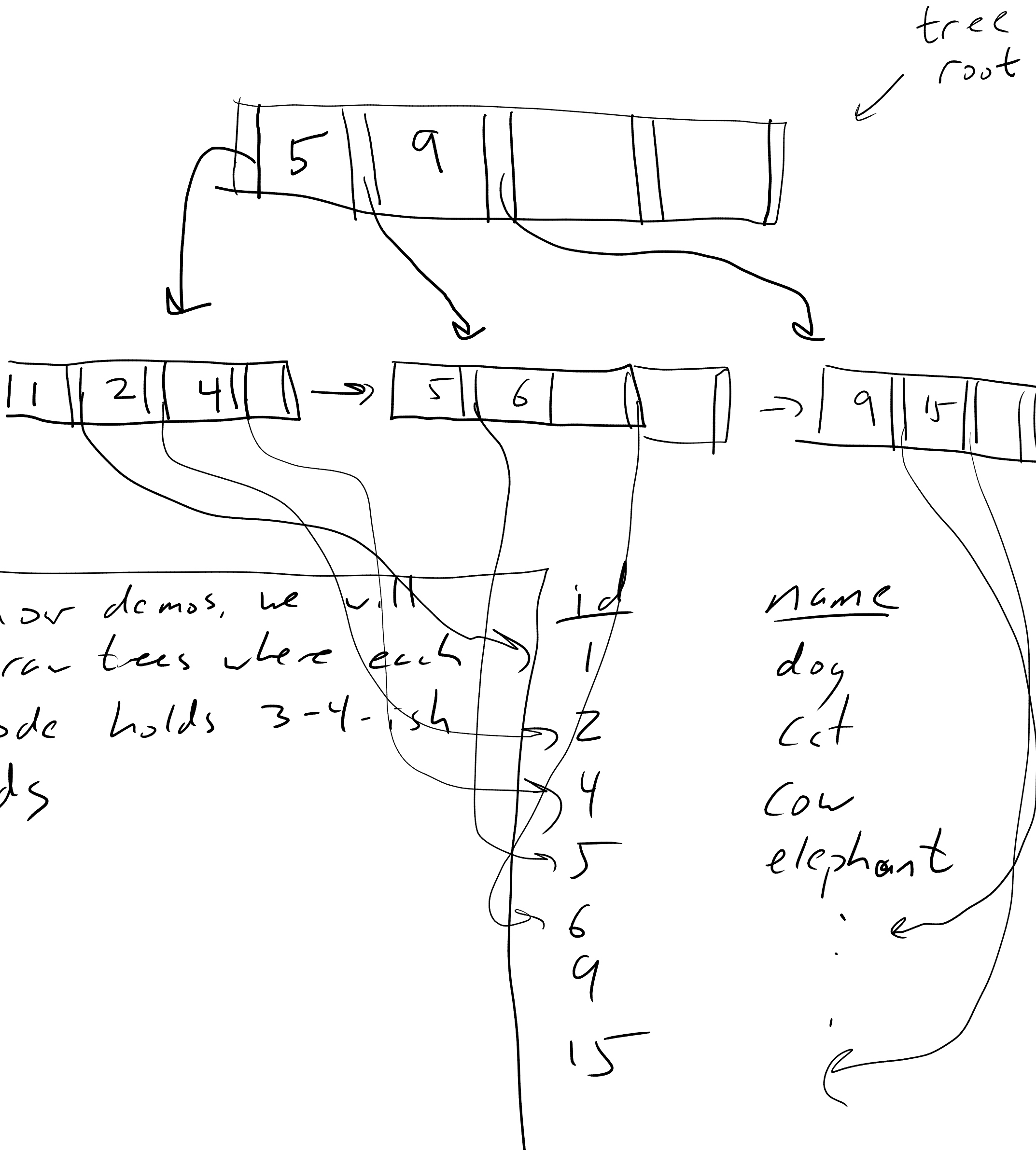
each rid is a page id and a slot id (if shorts, 4+4 = 8 more bytes)

total of 12 bytes for each id+rid

If page is 4K bytes, I can store

≈ 4000/12 ≈ 350-ish

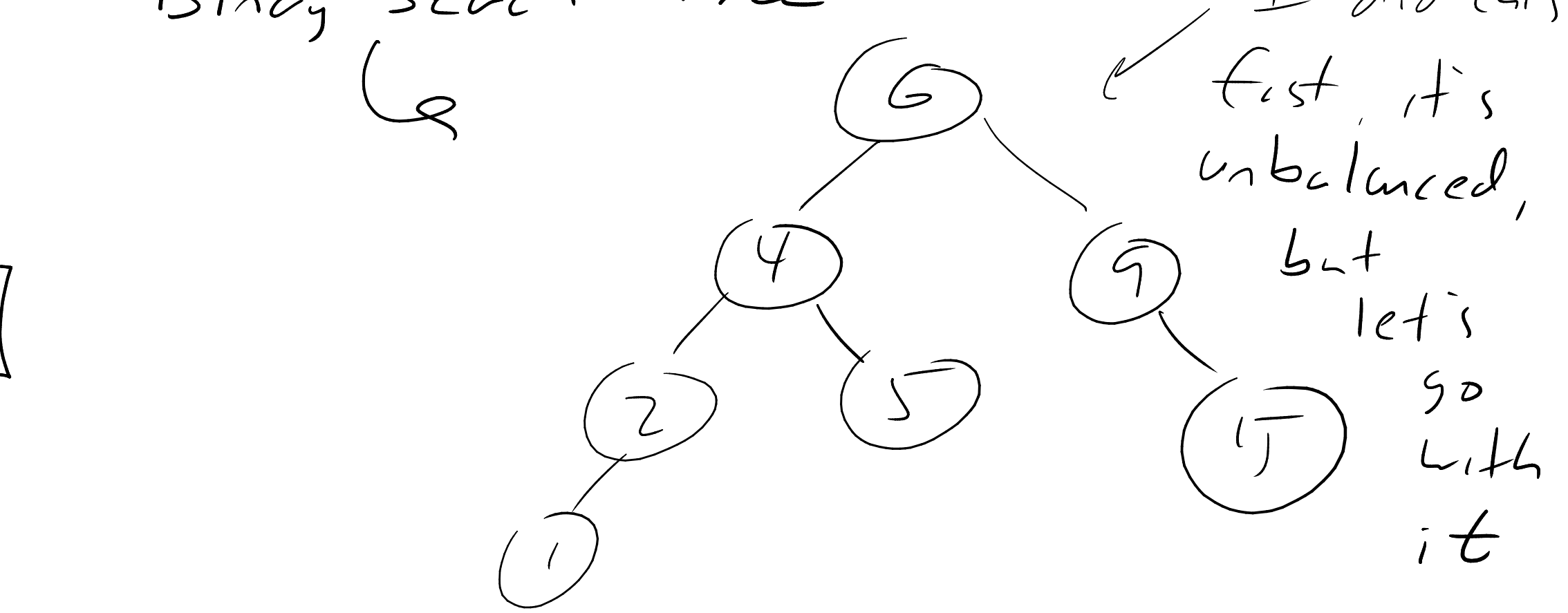
Example: each leaf holds a max of 4 keys+rids
internal nodes also contain 4 keys, (5 ptrs)



Intenal nodes contain n keys and n+1 ptrs to othe nodes

—

Before we cctally lookat how to insert, etc, why is this bette than a classic Binay Seach Tree?



I did this fist, it's unbalanced, but let's go with it

with a classic BST how many nodes do I hit to get to a leaf?

n keys in the tree

Approximctely $\simeq \log_2 n$ (± 1)

For B+ tree case

Approximctely $\simeq \log_b n$

b = branching factor = # of children at each node

When b is big,

$$\log_b n \ll\ll\ll \log_2 n$$

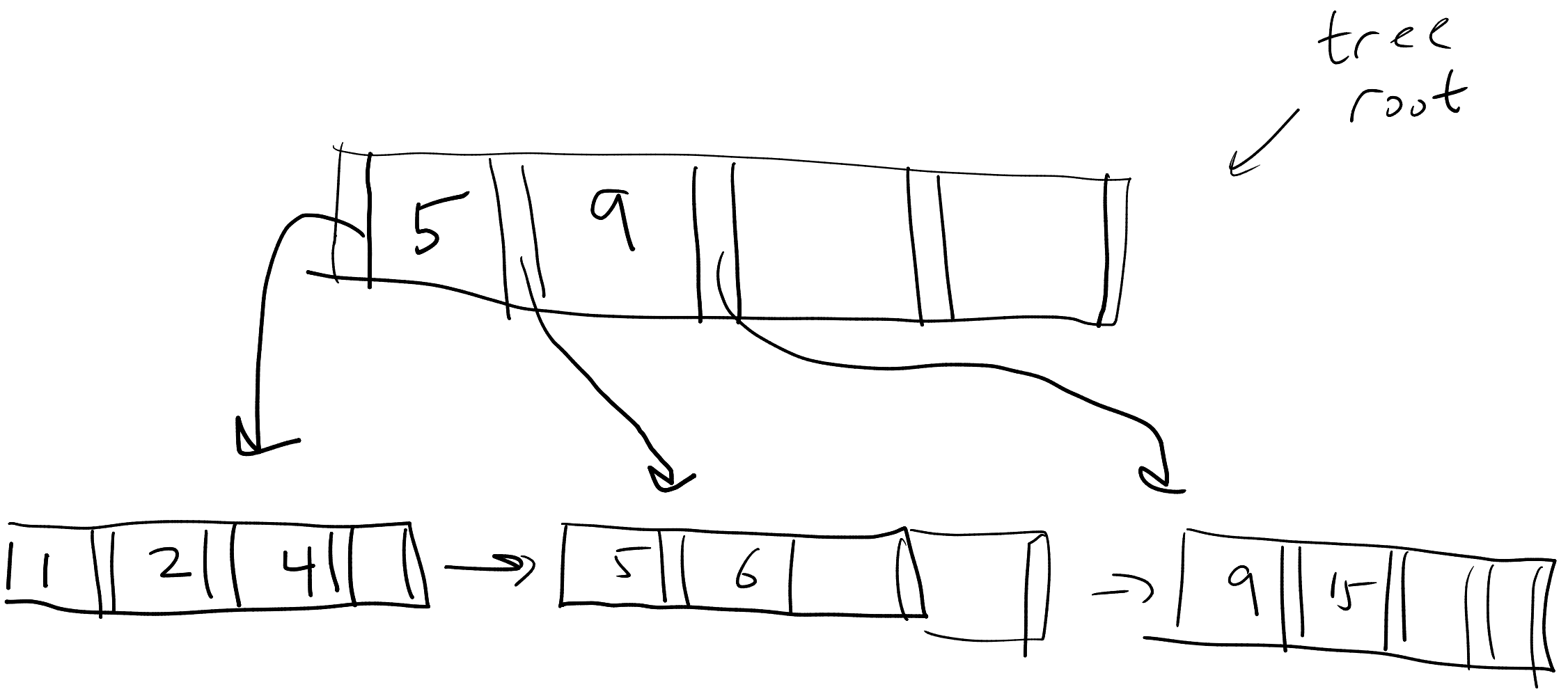
$\frac{n}{64}$	$\frac{\log_2 n}{6}$	$\frac{\log_8 n}{2}$
128	7	2.----
256	8	3

How many children do typical in a real example?
4k pages, maybe 12 bytes each entry $\approx$ 350 entries per page.
Let's round down to 100 entries per page.

How many nodes p-level?

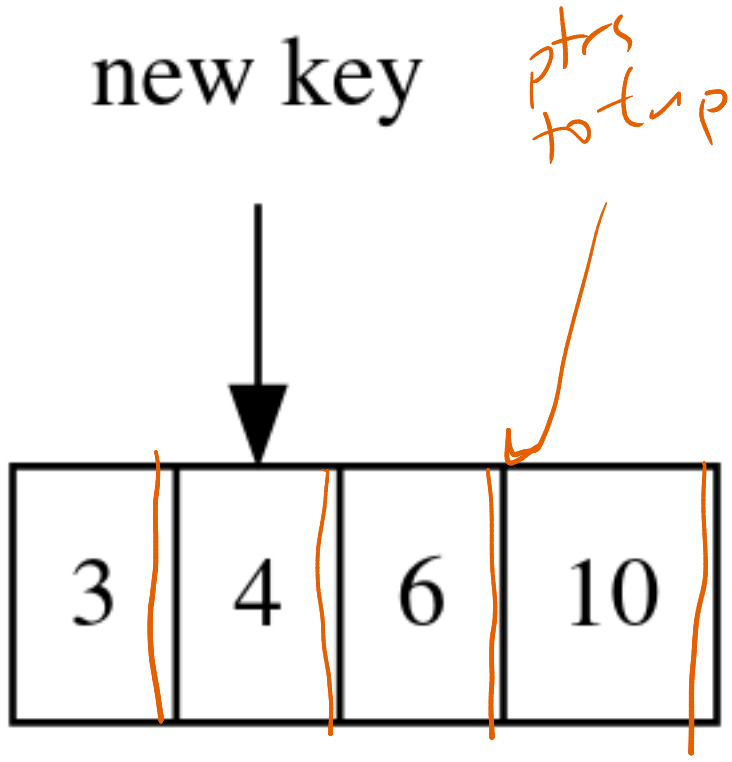
At root level (0)	1
1	100
2	$100^2 = 10,000$
3	$100^3 = 10^6 = 1$ million
⋮	
6	$100^6 = 10^{12} = 1$ trillion

I can find one id out of a trillion is 6 opections with a BST, lots more.



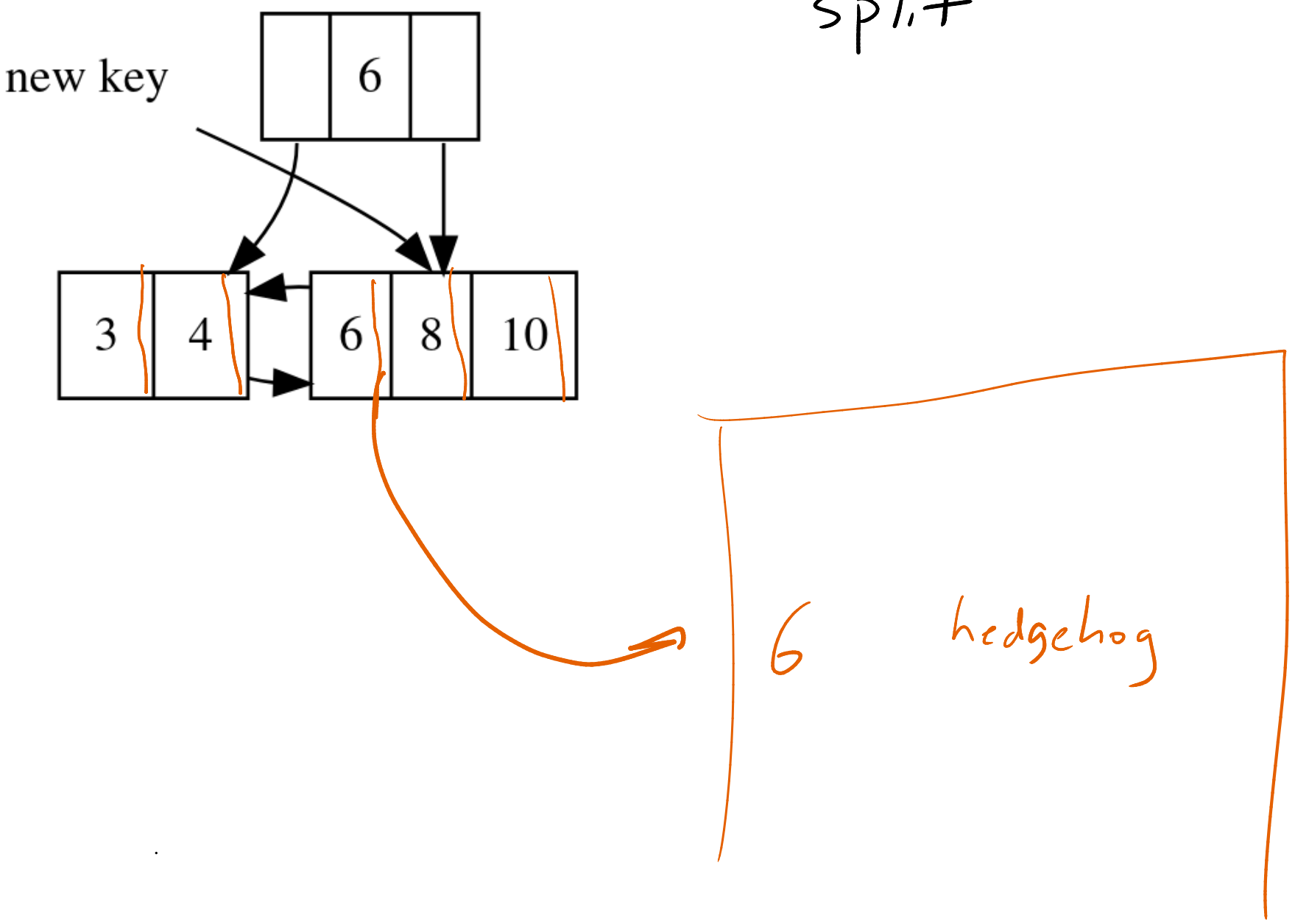
But we need to search within each node to find the key, or the keys to go between [binary search]
↳ all in memory calculations after having read the page.
Each new node you grab is a page retrieval from storage [slowwwwwwwwwwwww]
but looking up within a node is in memory and very fast

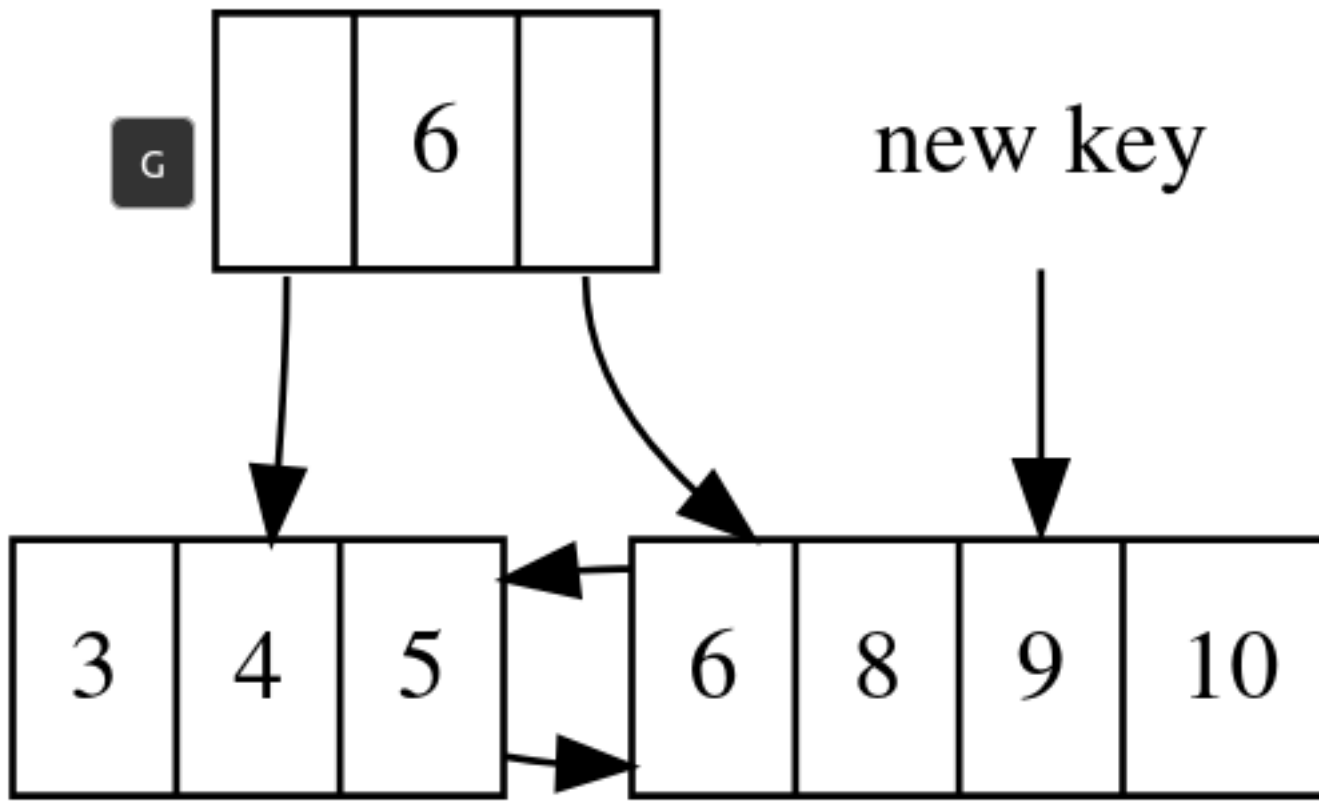
https://roy2220.github.io/bptree/visualization/#4



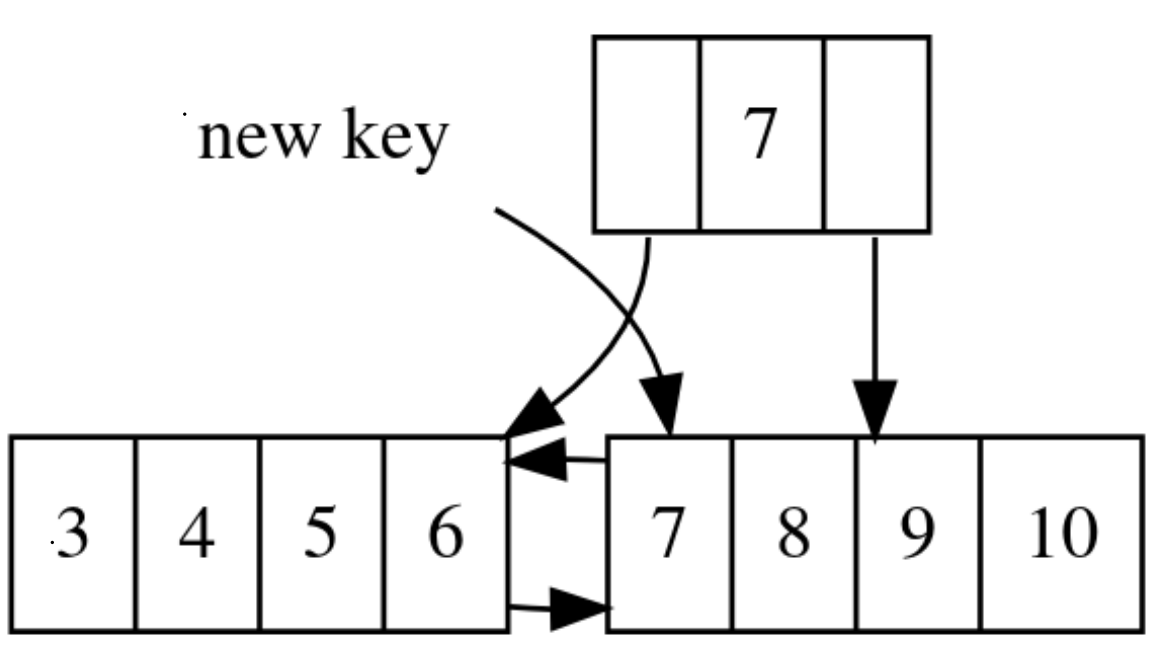
ptrs to tuples

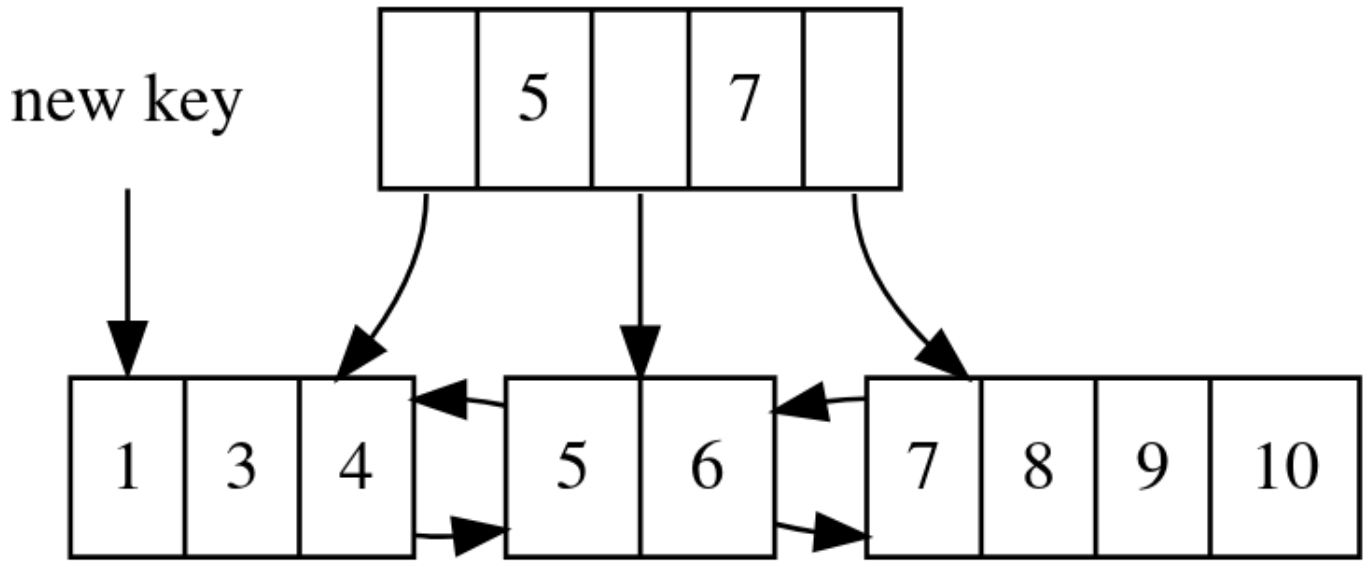
When we insert into a leaf, we split the leaf, and copy up a value (leftmost value of right node from split





(optionally, can do an optimization where you redistribute values to avoid a split)





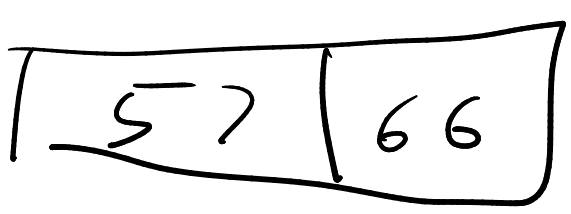
insert 70, but full, so split

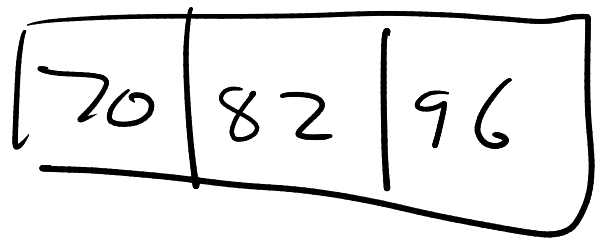
new key

Insert 82

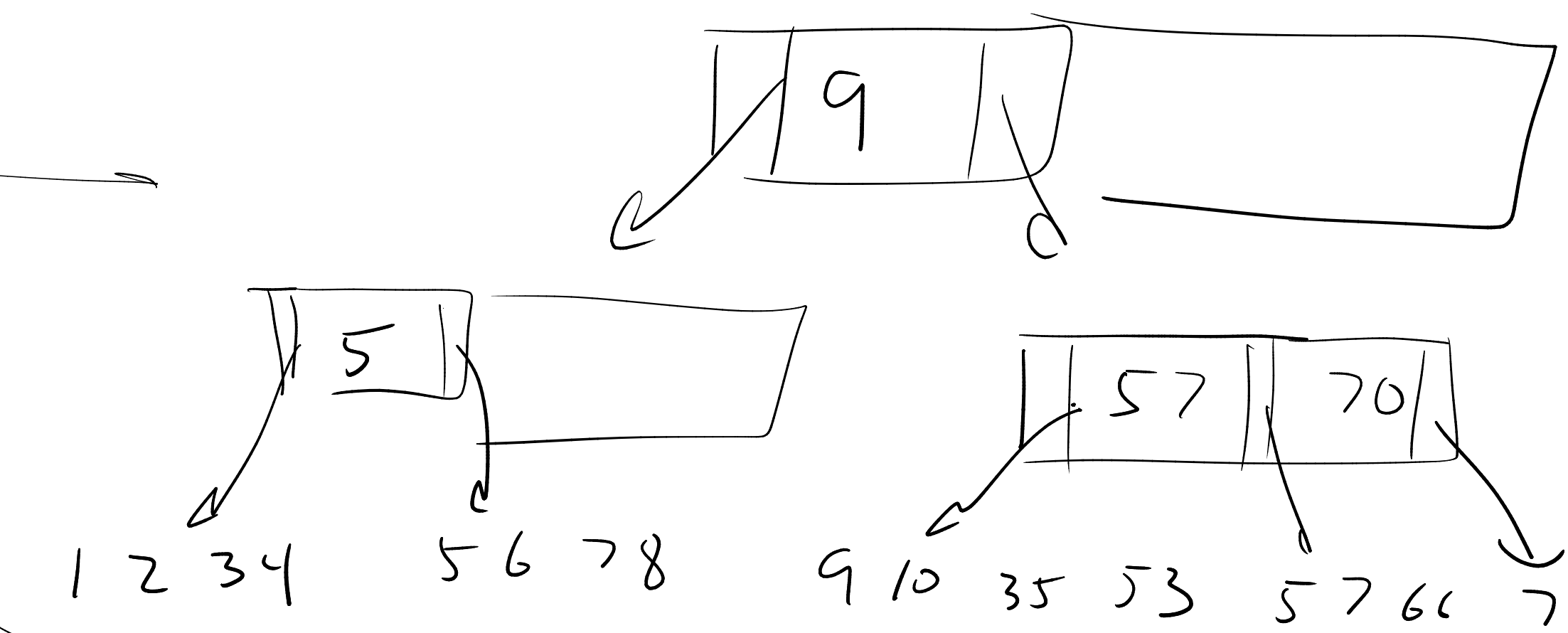
split

copy up 70





When an internal node splits, we move/push up the middle value (not a copy).



Q. precisely where do these arrows go?

Remember each node is a page, on disk

These "ptrs" are really just page ids

The leaves are all pointing to each other other in a linked list

B+

select * ..... where id ≥ 5 and id ≤ 50

Go to 5 in leaf → then itself to the right